Hamar Reader: Checked Summaries of Municipal Notices, and What the Checks Miss

Group 13
KIUA-2003 Applikasjon av kunstig intelligens/maskinlæring II
University of Inland Norway, Hamar

Abstract

Hamar kommune publishes hearings, plan notices, grant calls and council decisions on its *Lokaldemokrati* page. The page has no feed, no e-mail alerts and no API, it is written only in Norwegian, and articles disappear when they expire. We build a harness that polls the page, extracts each new article, removes personal data, summarises it with a small local language model in the reader's language, and checks every summary with code before anyone reads it, sending failures back to the model with the reasons. The walking skeleton runs end to end and feeds a phone app. We define success on two levels: form and safety checked by code on every run, and facts checked against a hand-made answer key. Three runs on the live page show why both are needed. A hand check of the third run found every deadline right, but the responsible body wrong in three of ten summaries and one summary in the wrong language; only four of ten were free of unsupported statements. We turned the two repeatable errors into code checks. With them in the retry loop, a fourth run passed 8/10 on form but only 3/10 on facts: a better score on the checks did not give better facts.

Repository: `https://github.com/Spaghetti221/Innlandet-Harness`. The READMEs explain how to reproduce the runs and the app.

1 Problem

The recurring need. The *Lokaldemokrati* page[1] is where Hamar kommune announces things residents can act on: hearings, public inspection and start of zoning plans, grant schemes, and council decisions. Keeping up is manual work every week: open the page, find what is new, read the Norwegian text and note the deadline. Articles are taken down when they expire (16 listed on 22.10, 11 on 03.11), so a resident who looks once a month can miss a notice entirely.

What exists already. The page offers no RSS feed, e-mail subscription or notification, only social media links, and nothing in other languages. Plan notices reach affected landowners directly, but not tenants, other residents or associations. Following the lecturer's advice to build on what exists (Meeting 5), we add what the page lacks: a push, a summary in the reader's language, the deadline as a reminder, and a check of every summary against its source.

Who has the need, and what the workflow produces. Residents who want to comment on plans near them, international residents and students who read English better than Norwegian, and local associations. For every new article the workflow writes one Markdown file with the title, the publish and expiry dates, a link to the original, a 3–6 bullet summary in the chosen language and links to the saved source; an export step feeds a phone app.

[1] `https://www.hamar.kommune.no/lokaldemokrati/`, checked 04.11.2026.

Preprint. Under review.

Fit with the constraints. The source is public, and its `robots.txt` (Koster et al., 2022) allows crawling with a `Crawl-delay` of 5 s, a non-standard directive the harness obeys. The page author is never read; e-mail addresses, phone numbers and names are replaced before text is stored or reaches the model, and the raw HTML is not kept; articles whose title names people are rejected. The model runs locally (Ollama, 2026; Meta, 2024), so no article text leaves the machine.

2 Definition of success

Small models produce fluent summaries that are not supported by their source (Maynez et al., 2020), so we separate what code can decide from what it cannot.

Level 1: form and safety, checked by code on every run. `python -m harness.hamar_news.check` exits 0 only if every summary passes all checks in Table 1, each decided by comparing the summary with the saved source page. The same checks run inside the pipeline right after each model answer.

Table 1: Level 1 checks, applied to every summary.

Check	Passes when
One summary per article	`state.json` and the summary files agree
Header and link	header complete; the link is the article's own URL
Dates match source	Published and Expires equal the dates in the saved source
Summary shape	3–6 flat bullet points, none repeated
No contact details or names	no e-mail, phone number or name (three rules, Section 3)
No invented dates	every date in the summary also appears in the source
Deadline kept	a date after *frist*, *innen* or *senest* is in the summary
Language	as requested, by counts of common English and Norwegian words
Responsible body	not "Hamar kommune" acting when the source names another body

Level 2: facts, checked against an answer key. For each article in a fixed test set we write by hand who is responsible, what is happening, where, what residents can do, the deadline and the case number. A summary passes if every statement about these is supported by the key. Our target for the exam is no unsupported statement about the deadline or the responsible body, and at most one summary in ten with any other unsupported statement. Level 2 is scored by hand today; for the exam we plan an LLM judge (Zheng et al., 2023) on top, measured against our hand scores, with its prompt and failure modes documented. A run is successful only if both levels pass. When we first scored run 3 by hand, it met neither target (below).

Runs on the live page. Table 2 scores all runs with the final Level 1 check; the prize article, rejected from run 2 on, is left out of run 1. Run 2 (stricter prompt, glossary, names removed before the model) fixed names and the translation of *pumpestasjon*, but kept nested lists and, as the later checks show, began to name Hamar kommune as the actor where another body acts. Run 3 added retry with feedback, which fixed the format. Each variant ran once on ten articles with a non-deterministic model, so these are observations, not measured effects.

Table 2: Runs on the live page (Llama 3.2 3B, English). Level 1 uses the final check for all runs; Level 2 is our hand score; time per article includes the 5 s pause between requests.

Run	Change before the run	Level 1	Level 2	Calls	s/article
1	first version	5/10	–	11[a]	6.7
2	stricter prompt, glossary, names removed before the model	2/10	–	10	9.1
3	retry with feedback (at most 3 attempts), deadline check	6/10	4/10	17	12.3
4	language and responsible-body checks in the retry loop (05.11)	8/10	3/10	20	13.4

[a] Includes the prize article, which later runs reject.

What the hand check of run 3 found. All ten deadlines were right, but twelve statements were unsupported: three summaries named Hamar kommune as the actor where the county, a consultancy or the national agency Bufdir acts; one was in Norwegian; one claimed an impact assessment the source rules out (Level 2: 4/10). Tracing the actor error: the extracted text named the right body, the model defaulted to the municipality whose page it read, and no check compared who acts. Both repeated errors became Level 1 checks (Table 1); with them run 3 scores 6/10. Every attempt is now stored.

Run 4 (05.11). With the two new checks in the retry loop, retries fixed the language (1/1), the format (4/4) and the actor in one of three summaries. In the other two the model added the right body elsewhere but kept the wrong first sentence through all three attempts: the feedback names the problem, not the sentence. The hand check fell to 3/10. Every deadline was right, but new errors appeared (*startlån*, a housing loan, became "start-up loans for new businesses"), and the stored attempts show one error that a retry introduced: "the council meeting can be attended" appears only in the second attempt. Raising the Level 1 score did not make the facts better.

3 Architecture

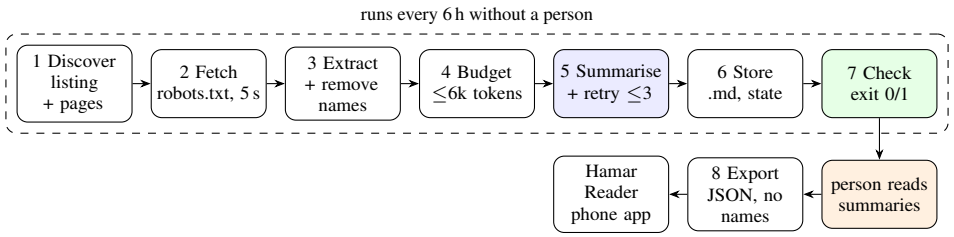


Figure 1: Stages 1–4 and 6–8 are plain code; stage 5 is the only model call (Llama 3.2 3B), repeated with the failed checks as feedback, at most three times. A person reviews before anything is exported. `state.json` makes the loop idempotent: an article is marked done only after its summary is written, so a failed call is retried on the next run.

Stages. Each stage is a tested module, and stages hand over files on disk. *Discover* keeps only article links; *Fetch* sends an identifying User-Agent and obeys the crawl delay. *Extract* accepts only pages marked as single articles, keeps title, lead and body, and removes contact details and names by three rules: a known first name followed by a surname; capitalised words after phrases such as "rettes til" or "kontaktperson"; capitalised words before a contact detail. An article whose title names people is rejected and remembered only as a hash. *Budget* cuts text above 6,000 tokens and flags it (the longest article is about 1.2k). *Summarise* calls Llama 3.2 3B through Ollama (temperature 0.2, context 8,192); a deterministic mock replaces it in tests. *Store* writes the cleaned source as JSON (never the raw HTML), one summary file per article, every model attempt, an index and the state; *Check* runs Level 1; *Export* writes the app's data file.

Context the model receives. Only a system prompt kept as a versioned context file (`prompts/summarise_system.md`) and the cleaned article inside `<article>` tags. Its eight rules cover the language, at most six flat bullets, deadlines and case numbers copied exactly, facts from the article only, no names, and treating the article as data, not instructions. English adds a glossary of sixteen planning terms. On a retry the model also gets its previous answer and the list of failed checks. Nothing else is in the context, so calls cannot drift over a long run.

Digital prototype. Following the lecturer's advice we built Hamar Reader, a phone app built with Expo (Expo, 2026), that shows the summaries in English and Norwegian, labels each notice from keywords in its Norwegian title, and sets a reminder three days before the deadline. It reads only the exported file and calls neither the site nor the model.

Technical minimum. Checkable output (Level 1, exit 0/1); eight stages, one of which calls the model with feedback; tools (HTTP, files, a model server, shell); and two loops that run without a person (polling every six hours, retry).

Planned for the exam. Stage 5 becomes three: *classify* the notice; *extract* who, what, where, deadline and case number as JSON, each value quoting its evidence, which code checks against the source with retry; and *summarise* from the verified fields only. Extracting *who* with evidence is aimed directly at the actor error above, and retry feedback will quote the failing sentence.

4 Schedule

The demonstration is in week 46 or 47 (slot on Canvas). We plan backwards from the earliest slot (09.11), freezing the demo version on 08.11; a week 47 slot gives an extra week of stress testing.

Table 3: Milestones, planned backwards from the demonstration. A and B are the two group members.

Date	Who	Milestone (done when)
03.11	A	Skeleton, three real-model runs, retry loop, Hamar Reader app
05.11	both	Hand check of run 3; language and actor checks; no raw HTML; all attempts stored; run 4
06.11	both	**Compulsory 2 submitted** (retake)
08.11	B	**Demo frozen:** test set with answer keys; one command scores Level 1 and 2
W46–47	both	**Live demonstration** with the app; rehearsal the day before
15.11	B	Classify and evidence-checked extract stages; LLM judge measured against hand scores
20.11	A	Kill switch, token budget; two stress tests (prompt injection; outage) with regression
27.11	both	Demo feedback addressed; white paper draft; at least eight meeting reports
03.12	both	**White paper submitted**; repository public; commit tagged `exam`

5 Risk assessment

Table 4: Most likely failures and our response, most serious first. L = likelihood, I = impact.

Risk	L	I	What we do when it happens
The model states wrong facts or acts outside its instructions (observed: wrong body 3/10, wrong language 1/10)	H	H	Nothing is exported without a person reading it. Evidence-checked extraction and Level 2 scoring; if the error rate stays high, compare with a larger model (local, or the Claude API under a token budget).
A personal name reaches a summary (observed 03.11)	M	H	Three removal rules before the model, no raw HTML stored, rejection of articles about named people, a name check, and an export that skips any summary with a name. We count misses on the test set.
The site changes layout, so nothing is found	M	H	Unknown pages are rejected. Kill switch: stop after two runs in a row with no articles, or when more than a third of summaries fail Level 1. The demo can replay saved pages.
Automated reading is not welcome: `robots.txt` blocks ClaudeBot and the terms are silent	M	H	Own User-Agent, `Crawl-delay` obeyed, one poll every six hours, only short summaries with links. We ask the lecturer and, if needed, the municipality; if the answer is no, we use saved snapshots.
Ollama is slow or missing on the demo laptop	M	M	6.7–12.3 s per article measured, including the 5 s pause; pre-run outputs and the mock as fallback.
Prompt injection inside an article	L	M	Article wrapped as data; the prompt says to ignore instructions in it; planned stress test.

References

Expo. Expo documentation. `https://expo.dev`, accessed 04.11.2026.

M. Koster, G. Illyes, H. Zeller, and L. Sassman. Robots Exclusion Protocol. RFC 9309, IETF, 2022.

J. Maynez, S. Narayan, B. Bohnet, and R. McDonald. On faithfulness and factuality in abstractive summarization. In *Proc. ACL*, 2020.

Meta. Llama 3.2 3B Instruct model card. https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct, 2024.

Ollama. https://ollama.com, accessed 04.11.2026.

L. Zheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *NeurIPS Datasets and Benchmarks*, 2023.

A Prompt logs (excerpts)

22.10, member B: choosing the problem. *Prompt (condensed):* "Read the Compulsory 2 section and find two real and relevant community problems in Hamar, compare them with a third problem I've thought of, rubbish collection, and identify the one with the most potential. Do not write any code yet." *Agent:* proposed a municipal hearings digest (P1) and a deduplicated events digest (P2), and compared them with rubbish collection (P3) on recurrence, existing solutions, source access, stages, checkability and reject cases. It recommended P1. *Human decision:* not accepted directly; the choice was left to the pair, and in the meeting on 29.10 we noted that we could not yet confirm that automated access to the municipal site was allowed.

29.10, member B: building the skeleton. *Prompt (condensed):* "Fetch new articles from hamar.kommune.no/lokaldemokrati, save the HTML, strip all but the article text, and feed it to a local Llama 3.2 3B for summarisation. Do not install or run a model here; add a mock mode. Let the user choose and save the summary language. Check every six hours, but make it easy to test with seconds. Summarise each article only once, one summary per file, with links to the original. There is no 'article/' prefix, so you need a filter. What context window is needed?" *Agent:* inspected the live site, found that the footer reuses the article-body attribute for contact boxes and scoped extraction to the main article element, dropped the author field, added scrubbing of contact details, wrote the system prompt, mock mode, profiles, and 19 tests. It measured articles at 140–510 words and recommended a context of at least 4,096 tokens (8,192 configured). *Human decision:* nothing committed until reviewed.

03.11, member A: running it and starting C2. *Prompt (condensed):* "Help me open and run this in PyCharm", followed by the error `No module named 'bs4'`, then "start C2". *Agent:* fixed the environment, read the brief and the code, and checked `robots.txt`. *Correction found:* the site sets `Crawl-delay: 5` but the code waited 2 seconds; the fetcher now obeys the site's delay (with a test). It added the output check and its tests (including failing cases), moved the system prompt into a context file, and drafted `SPEC.md` and this report. *Raised for human decision:* names of prize winners in a stored title, and the site's block on `ClaudeBot`. Later the same day we ran the real model, and the agent read the failing summaries and proposed the repairs described in Section 2. It also found and corrected its own check, which had accepted an indented sub-list. After the second run it claimed that one summary had lost its deadline; this was wrong (it had read lines cut at 170 characters) and was withdrawn after a direct check. Asked why we use Ollama and not Claude, it suggested supporting both and comparing. *Human decision:* not yet made; the pair decides before 05.11.

04.11, member A: evaluating the report. *Prompt (translated):* "Evaluate the report against the assignment text and the grading criteria", then "Do it, and improve the report based on the evaluation". *Agent:* found that the runs table compared pass rates measured with three different versions of the check, rescored the archived runs with the final check (5/11, 5/10, 9/10 instead of 6/11, 5/10, 10/10), pointed out that our definition of success contradicted our own finding of a wrong-fact summary that passed, and checked that the municipal page offers no feed or e-mail alerts. *Human decision:* we accepted the two-level definition of success; the Level 2 targets and the owners in the schedule are ours to confirm.

04.11, member A: the better prompts, sent for real. After the sessions above we wrote better versions of our prompts and sent them. The key one: *"Run 3 passed 10/10. Do not trust the number. Read every summary in full and compare each statement with the source text: who is responsible, what is happening, where, and when. List every statement that is not supported by the source, even if all checks passed." Agent:* found twelve unsupported statements, the wrong responsible body in three summaries and one summary in Norwegian (Level 2: 4/10), and that the raw HTML we stored still held names. *Human decision:* turn the two repeated errors into code checks, stop storing raw HTML, store every attempt, and report the hand score instead of the check's 10/10.

B Meeting reports

Meeting 1 (week 40, 01–02.10). Created the shared repository and scaffolded it with Claude Code in plan mode from the lecture prompt: placeholders for `README.md` and `SPEC.md`, `CLAUDE.md` with the project rules (never edit `eval/` without asking, no personal data, secrets only in `.env`), the folder structure, and `.claude/settings.json` denying reads of `.env`. Discussed the brief: finding the problem is part of the assignment.

Meeting 2 (week 42, 15.10). During the study week we switched the first probe source from Entur to the MET Norway forecast for Hamar, and specified task 001 with a failing check in `eval/` and edits to `eval/` denied. At the lecture we discussed problem definitions and got guidance from the lecturer on the project and on working with the coding agent. Decision: tighten the agent's permissions so it cannot read secrets, change its own settings or edit the checks. Open: task 001 not implemented, no shortlist of problems.

Meeting 3 (week 43, 22.10). Discussed problem definitions ahead of Compulsory 2. The same day the coding agent compared three candidates (municipal hearings digest, a weekly events digest, rubbish collection) and recommended the hearings digest. We did not accept it directly: we wanted to choose together and first check whether automated access to the municipal site was allowed. `SPEC.md` stayed empty until then.

Meeting 4 (week 44, 29.10). Compared candidates against the brief and the grading: municipal hearings digest (strong, but no feed and access not confirmed); hazard digest from NVE and MET (built and tested, then dropped because all warnings were green, so the demo would be mostly synthetic); public transport disruptions from Entur (open data, but a narrower need with little for the agent to judge); rubbish collection (largely solved already); jobs, tenders and fixtures (personal data or unclear terms); meeting places against loneliness (no machine-readable sources). No topic chosen yet. Built a prototype for an events candidate in which extraction agents must quote evidence for every date, time and place, checked by code with up to three attempts; 4/4 test cases as expected, and one case of model variance on start times.

Meeting 5 (week 45, 03.11, supervision with the lecturer on Teams, about one hour). We went through our candidates: public transport disruptions, hazard warnings (built, then dropped), the culture calendar prototype, which we think compares well with existing calendars, and the Hamar *Lokaldemokrati* summariser. What we took away: build a digital prototype that can be shown; give the demo and the report one storyline (problem, user, run, check, what broke, how we fixed it); and we may build on top of existing services, which revises our week 44 criterion of dropping candidates that were "largely solved already". *Decision:* we continue with the news app for local politics as our Compulsory 2 project. Status the same day: three real-model runs, repairs, a retry-with-feedback loop, and in the evening a first version of the Hamar Reader app as the digital prototype. *Actions:* third run with the retry loop (member A, 05.11); decide whether to add the Claude API (both, 05.11); rewrite the report in our own words (both, 05.11); submit (06.11); plan the demo storyline around the app (08.11).